

Dia# 1

Agentes de IA organizado por Google X y Kaggle

Bienvenidos a “Deep Dive”. Hoy vamos a entrar de lleno en un tema muy emocionante: la arquitectura de los agentes de IA. Así es, nos vamos a enfocar en el whitepaper del primer día de ese intensivo de cinco días sobre agentes de IA organizado por Google X y Kaggle.

Realmente, esto se siente como la guía esencial para cualquier persona que quiera construir soluciones con IA generativa, yendo más allá de simples demostraciones para crear sistemas robustos y listos para producción.

Hablemos de ese cambio de paradigma. ¿Cuál es la transformación clave aquí? Pues es fundamental. Si pensamos en la IA históricamente, era en su mayoría pasiva: respondía preguntas, traducía, reaccionaba a un prompt. Ahora hablamos de agentes autónomos, orientados a objetivos. Estos agentes no solo conversan, sino que planean, actúan y resuelven problemas complejos en múltiples pasos, sin que nadie los guíe en cada momento. Esa es la parte de autonomía: ejecutan acciones en el mundo para alcanzar una meta.

El whitepaper desglosa la anatomía de un agente en tres partes principales: el modelo, que es como el cerebro; las herramientas, que actuarían como las manos; y la capa de orquestación, que es el director de toda esta sinfonía. Empecemos con el modelo, el cerebro. Es el LLM, ¿verdad? ¿Cuál es su función específica en un agente? Su papel principal es gestionar la ventana de contexto. Es decir, decidir constantemente qué información es relevante en cada instante, ya sea proveniente de la misión, la memoria o lo que acaban de hacer las herramientas. El modelo filtra todo eso y determina qué información necesita para el siguiente proceso de razonamiento.

El modelo razona, pero necesita de herramientas para realizar acciones en el mundo, ya sean APIs, funciones de código específicas, acceso a bases de datos, etc. El agente puede, por ejemplo, buscar datos de clientes o verificar inventario. Además, el modelo decide qué herramienta es necesaria en cada paso de su plan, y la capa de orquestación es la que realmente hace esa llamada. El resultado de la herramienta vuelve directamente al contexto del modelo, listo para la siguiente ronda de razonamiento.

La capa de orquestación hace mucho más que correr código. Es la gestora del proceso completo: maneja el ciclo operativo, la planificación, el seguimiento de la memoria o estado y ejecuta la estrategia de razonamiento. Esa estrategia puede incluir métodos como “chain of thought” o “react”, que mezclan razonamiento y acción: el agente piensa, decide qué acción tomar con base en su meta y la información actual, ejecuta la acción usando una herramienta, observa el resultado y vuelve a razonar con esa nueva información. Así se repite el ciclo, haciendo que el modelo pase de ser solo un generador de texto a convertirse en un ejecutor capaz de realizar tareas complejas.

Supongamos una tarea simple, como organizar un viaje de equipo. El agente recibe la misión, revisa sus herramientas (acceso al calendario, APIs de reservas), consulta lo que tiene en memoria y entra en la fase de planificación: por ejemplo, primero necesita la lista del equipo, así que usa la herramienta correspondiente. Después, ejecuta esa acción, recibe la información, la integra en su memoria y vuelve a pensar: “Ya tengo la lista, ahora verifiquemos la disponibilidad”, y así sucesivamente hasta completar la misión. Este bucle “pensar, actuar, observar” se repite para cualquier tarea, desde organizar viajes hasta responder consultas de soporte.

El whitepaper introduce además una taxonomía de niveles de capacidad del agente. Nivel cero es solo el modelo de lenguaje, sin herramientas: no tiene acceso a información en tiempo real. Nivel uno ya conecta con el mundo mediante herramientas; puede buscar información actualizada, acceder a bases de datos, etc. Nivel dos es el solucionador estratégico de problemas: maneja tareas más complejas en varios pasos y gestiona activamente el contexto, por ejemplo, encontrando una cafetería a mitad de camino entre dos direcciones usando múltiples herramientas en cadena. Nivel tres es el sistema multiagente colaborativo, donde hay agentes especialistas que trabajan en equipo y pueden delegar tareas: no es solo una función compleja, sino que se le transfiere autonomía al agente receptor para que resuelva sub-tareas. Nivel cuatro es el sistema auto-evolutivo, capaz de detectar sus propias limitaciones y crear o adaptar nuevas herramientas o agentes según lo requiera la misión.

Para llevar estos agentes a producción y garantizar su fiabilidad, hay que considerar la selección del modelo. El modelo más grande no siempre es el mejor; a veces es preferible usar modelos especializados para tareas complejas y otros más rápidos o económicos para tareas sencillas. Así se optimizan los recursos y costos. En cuanto a las herramientas, se suelen utilizar bases de datos vectoriales para búsquedas, o APIs para ejecutar acciones concretas. Es fundamental que el modelo sepa exactamente cómo llamar a esas herramientas, con descripciones claras, tal como en un esquema OpenAPI.

La capa de orquestación define la personalidad y reglas de operación del agente, usualmente a través de prompts de sistema o “constituciones”, como decirle que nunca debe dar consejos financieros. También gestiona la memoria, que puede ser de corto plazo (datos temporales para la tarea actual) o de largo plazo (preferencias, interacciones pasadas). Esta memoria muchas veces se implementa como otra herramienta, usando bases de datos vectoriales que almacenan los recuerdos del agente.

Las pruebas y el debugging suponen un cambio radical respecto al software tradicional, porque la salida de un agente puede variar legítimamente aunque cumpla con la tarea. Por eso, se usan otros modelos para evaluar la calidad de las respuestas según un criterio detallado, así como herramientas de trazabilidad (telemetría) para registrar cada paso del razonamiento del agente. Cuando se detecta un fallo, ese caso se convierte en un test permanente, mejorando así el sistema de manera continua.

En cuanto a seguridad, se requiere una protección en capas: reglas de código duro para limitar acciones peligrosas, modelos de IA que revisen los pasos antes de ejecutarlos y, sobre todo,

dotar a cada agente de una identidad digital única y verificable, que delimite sus permisos y acceso a recursos. Esto se vuelve todavía más crítico cuando hay decenas o cientos de agentes interactuando: es esencial un punto central de control y monitorización.

Finalmente, los agentes deben evolucionar y mejorar constantemente, analizando sus experiencias, la retroalimentación de usuarios y cambios externos, optimizando prompts, ingeniería de contexto y creando o ajustando herramientas nuevas. El whitepaper menciona incluso la simulación, un "gimnasio" de agentes en entornos seguros de prueba, para simular interacciones complejas sin afectar a usuarios reales.

Como ejemplos concretos, se menciona Google Co-scientist, que es un sistema colaborativo de nivel tres o cuatro para investigación científica, con un agente supervisor que delega tareas a agentes especialistas, replicando flujos de trabajo reales. Otro ejemplo es Alphavolve, un sistema que usa la IA para descubrir y optimizar algoritmos, combinando generación de código y procesos evolutivos automáticos, con expertos humanos guiando y evaluando los resultados.

En resumen, construir agentes exitosos no consiste solo en escoger el modelo más avanzado. La clave está en la combinación del modelo para el razonamiento, las herramientas para la acción y la capa de orquestación que gestiona el ciclo completo. El éxito depende rigor de ingeniería, arquitectura, gobierno, seguridad, pruebas y análisis de observabilidad. El mensaje central es que el rol de los profesionales está cambiando: dejan de ser únicamente programadores para convertirse en arquitectos y directores de sistemas autónomos, listos para abordar tareas complejas y colaborar verdaderamente con nosotros.

Día #2 Herramientas de Agente e Interoperabilidad con el Protocolo de Contexto de Modelo (MCP)"

Las asignaciones se centran en la Unidad 2 titulada "Herramientas de Agente e Interoperabilidad con el Protocolo de Contexto de Modelo (MCP)". Como parte de la unidad, debes escuchar el

[podcast resumen](#)

, generado por

[NotebookLM](#)

.

Además, completa la lectura del

[whitepaper "Agent Tools & Interoperability with MCP"](#)

.

Los codelabs del día te guiarán para crear herramientas personalizadas para tus agentes, convirtiendo funciones Python en acciones utilizables. Utilizarás MCP y aprenderás a implementar operaciones de larga duración que permiten a un agente pausar llamadas a herramientas mientras espera aprobación humana para luego continuar.

Recuerda verificar tu número telefónico en Kaggle antes de iniciar, necesario para los codelabs, y consulta la guía de solución de problemas para resolver inconvenientes habituales.

Habrá un livestream mañana conducido por Kanchana Patlolla y Anant Nawalgaria, acompañados de invitados especiales, donde discutirán detalles de las asignaciones y compartirán perspectivas.

La página del curso es el centro principal donde encontrarás todos los recursos y actualizaciones del evento.

La comunidad anima a mantener un ambiente positivo y de apoyo, siguiendo las directrices comunitarias de Kaggle.

Este día avanza en enseñarte cómo tus agentes pueden utilizar funciones externas para ejecutar acciones o buscar datos en tiempo real más allá de su entrenamiento.

Aprenderás sobre MCP, sus componentes arquitectónicos, la capa de comunicación, riesgos y las brechas para su uso en empresas.

Es una unidad clave para entender la interoperabilidad y expansión de capacidades de tus agentes IA.

¡Feliz aprendizaje y nos vemos mañana!

The Kaggle Team

Esa es la transformación clave. Y el mapa para llegar ahí, cómo los modelos pasan de solo pensar a realmente hacer, es lo que vamos a analizar hoy. Sí, exactamente. Nuestro material de referencia es el white paper del segundo día de ese curso intensivo de 5 días sobre agentes de IA de Google y Kaggle. Realmente expone la arquitectura para convertir modelos en agentes activos. Y aquí es donde la idea de las herramientas se vuelve absolutamente central. Las herramientas son básicamente los sentidos y actuadores del agente. Son los ojos y las manos, como dijiste.

Bien. Pero ya sabes, históricamente conectar estas herramientas era un verdadero dolor de cabeza. Si tenías, digamos, n modelos diferentes y m herramientas o APIs diferentes. Oh, ya veo a dónde va esto. Sí. Terminabas con un problema exponencial $n * m$. Un desastre fragmentado de código personalizado, conectores hechos a medida para cada par. Simplemente no escalaba. Suena como una pesadilla. Honestamente, lo fue. Y la solución que la industria parece estar consolidando ahora es el protocolo de contexto del modelo o MCP. Surgió en 2024.

MCP. Sí, MCP. Está diseñado como un estándar abierto para simplificar toda esta integración. La meta es una manera unificada, casi plug-and-play, para conectar las herramientas, que fundamentalmente desacopla el agente, el cerebro, el razonador, de la herramienta específica que hace el trabajo, el actor.

Bien, eso suena prometedor. Desglosemos eso empezando desde el principio. Sí. Cuando decimos herramienta en este contexto de agentes AI, ¿de qué hablamos exactamente? Básicamente, una herramienta es solo una función o quizás un programa que una aplicación basada en LLM utiliza para hacer algo que el modelo mismo no puede hacer de forma nativa, ¿verdad? Y crucialmente, estas tareas caen en dos grandes categorías. O dejar que el modelo sepa algo nuevo, como obtener un pronóstico del tiempo en tiempo real usando una API o dejar que el modelo haga algo, como enviar un mensaje, actualizar una base de datos, hacer que algo suceda en el mundo exterior.

Entendido. Saber algo o hacer algo. Y la comunidad de desarrolladores, basada en el white paper, parece dividirlos en, ¿qué? Tres tipos principales. Así es. Tres categorías principales. Primero, están las herramientas funcionales. Bien. Son las que los desarrolladores definen explícitamente. Son funciones externas, tal vez código Python. Y a menudo usas cosas como docstrings detallados, especialmente en frameworks como el ADK de Google. Estos docstrings son críticos. Definen el contrato, las entradas y salidas entre el modelo y la función.

¿Verdad? Por ejemplo, puedes definir una función para ajustar los valores de luz en una casa inteligente, y el docstring especifica que necesitas brillo y color como entradas. Exactamente, el modelo lee esa documentación para entender exactamente cómo llamar a tu función correctamente. Todo se trata de ese contrato. Tiene sentido. ¿Cuál es el segundo tipo? El segundo son las herramientas incorporadas. Estas son diferentes porque las provee implícitamente el proveedor del servicio del modelo. Así que, como desarrollador, no ves realmente la definición de la herramienta. Simplemente funciona detrás de escena. Como la capacidad de Gemini para usar Google Search para fundamentar o ejecutar un poco de código por sí mismo. Ejemplos perfectos: fundamentación con búsqueda, ejecución de código, incluso obtener contenido desde una URL que proporcionas. Estas son capacidades integradas de la plataforma.

Y el tercer tipo, este suena muy interesante: herramientas de agente. Sí, es un concepto poderoso. Una herramienta de agente significa que en realidad estás invocando a otro agente separado como si fuera una herramienta. Bien, pero la distinción clave aquí es que no transfieres completamente la conversación o la tarea principal. Tu agente principal se mantiene al mando. Llama al subagente, recibe un resultado y después usa ese resultado como una entrada o recurso para su propio razonamiento. A menudo gestionado con algo como una clase de herramienta de agente en un SDK.

Así que es más como delegar que una transferencia completa. Exactamente. Es jerárquico. Piénsalo como un gerente que le pide a un equipo especialista un informe en lugar de simplemente enviarles todo el proyecto. Bien, lo entiendo. Entonces, si mi agente principal necesita, no sé, analizar algunos datos y luego enviar un resumen por correo, podría llamar a un agente de análisis de datos como herramienta, obtener el resumen y luego usar una función separada para enviar el correo. Exacto, compones capacidades.

Ahora, ampliando un poco, el white paper también habla de una taxonomía más amplia agrupando herramientas por lo que hacen. Correcto, como recuperación de información. Sí, obtener datos. Ejecución de acciones, que consiste en hacer cambios en el mundo real como enviar ese correo o hacer una reserva. Integración con APIs del sistema, obviamente, conectar con otro software, y muy importante, herramientas con intervención humana cuando el agente necesita detenerse y pedir permiso o aclaración a una persona.

Esa estructura ayuda, pero sabes que tener todos estos tipos de herramientas es una cosa, hacer que funcionen de manera confiable para que el agente no falle constantemente es otra. Parece el verdadero desafío. Esto nos lleva a mejores prácticas, que parece crítico para cualquiera que esté construyendo esto.

Totalmente crítico. Si estás desarrollando herramientas para agentes, estas no son solo sugerencias. Son esenciales para el éxito. Bien, cuéntanos. ¿Cuál es la regla número uno? Regla número uno, innegociable. La documentación es primordial. En serio, la única manera en que el modelo sabe qué hace tu herramienta, qué necesita y qué devuelve es a través de la documentación que provees. El nombre, la descripción, los parámetros. Esta información literalmente se alimenta en el contexto del modelo.

Así que la claridad es clave. Absolutamente clave. Usa nombres claros y descriptivos. El ejemplo en el paper es buenísimo. Crear un bug crítico con prioridad es infinitamente mejor para el modelo que algo vago como "actualizar en JIRA". Sé específico. El nombre y la descripción se vuelven básicamente el manual de instrucciones para el LLM.

Exacto. Lo que nos lleva al siguiente punto. Describe la acción, no el detalle de implementación. Necesitas decirle al modelo qué tarea debe cumplir. Crea un bug para describir este problema, no cómo llamar a tu función específica. Usa la herramienta crear bug. Ah, está bien. ¿Por qué esa distinción es tan importante? Si le dices el cómo, puedes confundirlo. El modelo podría simplemente repetir tu instrucción o quedar atascado pensando en el nombre de la herramienta en lugar del objetivo. Quieres que el LLM razone. "Necesito crear un bug" y la herramienta debe ser el mecanismo simple que ejecuta esa decisión. Deja que el LLM razone, deja que la herramienta actúe, ¿verdad? Reforzando esa separación de responsabilidades. El LLM es el cerebro, la herramienta es la mano.

Exactamente. Y relacionado con eso está publicar tareas, no llamadas API en bruto. Es tentador, especialmente con APIs empresariales complejas, crear solo un wrapper sencillo. Eso suele ser un error porque esas APIs pueden ser enormes, tal vez decenas de parámetros. Una buena herramienta debe encapsular una única tarea clara y de alto nivel que el agente necesite realizar, no exponer la complejidad en bruto de un sistema legado. Piensa en reservar una sala de reuniones, no llamar al endpoint complejo del calendario con 15 flags opcionales. Mantén el foco en la tarea y abstrae la complejidad subyacente.

Tiene sentido. Y ese foco probablemente impacta también la salida, ¿verdad? ¿Qué pasa con los datos que envía la herramienta? Punto gigante. Diseña para una salida concisa. Esto es vital. Si tu herramienta descarga, digamos, una hoja de cálculo enorme o un enorme archivo de logs y trata de meterlo todo en la ventana de contexto del LLM... Malas noticias. Muy malas noticias. Obtienes un bloat de ventana de contexto. Se comen tokens, lo que cuesta dinero. Aumenta la latencia. Y crucialmente, de hecho degrada la capacidad de razonamiento del LLM porque está revisando toneladas de datos irrelevantes.

Entonces, ¿cuál es la alternativa? No devuelvas los datos en bruto. Devuelve un resumen conciso o tal vez solo una confirmación o mejor aún, un referente, como un URI que apunta a dónde se almacenan los datos completos externamente. Servicios como el artifact service de Google ADK están contruidos para esto. Déjale saber al agente dónde están los datos. No se los sobrecargues.

Bien, mantén el contexto limpio. Tiene perfecto sentido. Una última práctica. Las cosas inevitablemente saldrán mal. ¿Cómo deberían manejarse los errores en las herramientas? Los errores suceden. Sí. Primero, usa validación de esquemas en entradas y salidas rigurosamente. Pero cuando ocurre un error durante la ejecución, el mensaje de error debe ser descriptivo y idealmente instructivo. No solo error 500. Definitivamente no. Un buen mensaje de error le dice al LLM qué salió mal y tal vez cómo recuperarse. Algo como: límite de tasa de API excedido. Por favor espera 15 segundos antes de llamar a esta herramienta otra vez. Eso le da al agente contexto y un camino claro a seguir. Convierte un fallo en algo potencialmente accionable.

Esas son pautas muy prácticas. Cualquiera que defina funciones de agente probablemente pueda usarlas de inmediato. Bien, cambiemos de tema ahora al propio protocolo de contexto del modelo, MCP. Esta es la capa de estandarización que busca hacer todo este uso de herramientas escalable e interoperable. Dijiste que se inspira en el protocolo del servidor de lenguaje LSP. ¿Cómo funciona esa interacción cliente-servidor? Correcto. Toma esa idea arquitectónica central. Básicamente tienes tres partes principales trabajando juntas.

Primero, está el host MCP. La aplicación principal, quizá aquello con lo que interactúa el usuario final. El host maneja la experiencia del usuario, orquesta el proceso de pensamiento del agente, decide cuando se necesitan herramientas, pero críticamente también es el aplicador de reglas; es responsable de aplicar cualquier salvaguarda o política de seguridad. Es como el policía de tránsito.

Entendido. Luego está el cliente MCP. El cliente está incrustado dentro del host. Piénsalo como el módulo de comunicación dedicado. Su trabajo es mantener la conexión con el servidor, manejar el ciclo de vida de la sesión y enviar los comandos para ejecutar herramientas según lo que el host necesite.

Y finalmente, el servidor MCP. Este es el programa real que provee las herramientas o capacidades. Su trabajo es anunciar qué herramientas ofrece, escuchar comandos del cliente, ejecutar esos comandos y luego enviar los resultados de vuelta. Esta separación es clave porque permite a los desarrolladores que construyen la lógica del agente en el host enfocarse solo en el razonamiento mientras otros equipos, quizá

terceros, se enfocan en construir herramientas especializadas, seguras y confiables en el servidor. Esto promueve la modularidad.

¿Cómo se comunican estos componentes? La capa de comunicación usa JSON RPC 2.0. Es un estándar bien establecido basado en texto. Mantiene las cosas relativamente simples e interoperables. La capa de transporte: MCP define dos formas principales. Para desarrollo local o cuando el servidor puede correr como proceso hijo, a menudo se usa STDIO, entrada y salida estándar; es súper rápido, comunicación directa, muy eficiente si todo está en la misma máquina. Pero para la mayoría de sistemas distribuidos reales, se usa HTTP con capacidad de streaming. Esto soporta eventos enviados por el servidor, así el servidor puede transmitir resultados de manera continua, ideal para herramientas que pueden tardar en ejecutarse.

El white paper menciona que MCP define varias primitivas como herramientas, recursos, prompts, pero realmente enfatiza que las herramientas son el evento principal. La mayoría del esfuerzo en implementación se ha focalizado en la definición y ejecución de herramientas.

Centrémonos en la definición de herramienta dentro de MCP. ¿Cómo hace el protocolo para imponer rigor allí? Usa un esquema JSON estandarizado. La definición de una herramienta debe tener campos como nombre y descripción. Y lo más importante, requiere un esquema de entrada que defina qué espera la herramienta, y opcionalmente un esquema de salida para lo que devolverá. El ejemplo del "obtener precio de acciones" muestra la entrada esperada como el símbolo de la acción, la fecha y la estructura de salida. Esto asegura que el host sabe exactamente cómo llamar a la herramienta y qué resultado esperar, sin conjeturas.

Los resultados generalmente son estructurados, es decir, objetos JSON que cumplen estrictamente con el esquema, que es lo preferido porque es fácil para el host y el LLM analizarlos y razonar con ellos confiablemente. El otro tipo son no estructurados, para cosas que no encajan en JSON, como texto bruto, audio, imágenes o referencias (URI) que apuntan a recursos externos, lo que ayuda a evitar sobrecarga de contexto.

MCP también formaliza los errores: hay fallos a nivel de protocolo JSON RPC, como llamar a métodos que el servidor no ofrece o enviar parámetros mal formados. Y errores durante la ejecución de la herramienta, por ejemplo que una API externa esté caída o no se encuentre el dato pedido; en ese caso, el servidor devuelve un objeto resultado con una bandera de error y un mensaje descriptivo que permite al LLM entender qué pasó y cómo potencialmente recuperarse.

Las ventajas estratégicas de la estandarización MCP son: acelerar el desarrollo y fomentar un ecosistema reutilizable. Al tener una forma estándar para que herramientas y agentes se comuniquen reduces significativamente la fricción y el tiempo para integrar nuevas capacidades, abriendo la entrada a más desarrolladores. Ya se habla de registros públicos MCP donde la gente puede publicar definiciones estandarizadas para sus servidores. Imagina un futuro donde puedas explorar y conectar fácilmente herramientas certificadas de diferentes proveedores, generando un efecto de red enorme.

¿Y para los agentes? Esto los hace más inteligentes o capaces. Permite capacidades dinámicas porque las herramientas se pueden anunciar y descubrir en tiempo real mediante el protocolo. Los agentes podrían encontrar y usar nuevas herramientas en ejecución sin necesidad de reprogramación explícita, aumentando su autonomía y adaptabilidad.

Además, ofrece flexibilidad arquitectónica significativa al desacoplar la lógica de razonamiento central de los agentes de las implementaciones específicas de las herramientas. Puedes construir sistemas más modulares. Se habla de crear una malla IA agentica, redes de agentes especializados y herramientas que se comuniquen vía MCP. Esto soporta diseños mucho más limpios.

Ahora, hablando de desafíos, el white paper reconoce que formalizar todo con esquemas detallados en MCP genera el problema de sobrecarga de contexto cuando un agente tiene acceso a cientos o miles de herramientas. Cargar todas las definiciones detalladas, nombres, descripciones, esquemas, en la ventana del LLM cada vez es completamente inviable. Se alcanza el límite de contexto, los costos se disparan y lo peor, el LLM se satura y lucha para decidir cuál herramienta es la más adecuada en la tarea actual disminuyendo la calidad del razonamiento.

Entonces, la estandarización crea esta potencial avalancha de herramientas, pero el LLM solo puede "beber" de una parte pequeña. ¿Cómo se cruza esa brecha? La estrategia más prometedora es aplicar un enfoque estilo rag (retrieval augmented generation), pero para herramientas. En lugar de cargar todas las definiciones posibles de herramientas, la idea es que cuando el agente necesita una, primero realiza una búsqueda semántica rápida en una base de datos indexada de herramientas disponibles. Esta búsqueda identifica solo las 3 a 5 herramientas más relevantes para la tarea inmediata. Solo se cargan las definiciones de esas candidatas para el paso final de razonamiento. Se considera el camino más viable para escalar a grandes cantidades de herramientas.

Finalmente, MCP por sí solo no tiene seguridad empresarial robusta, sin autenticación fuerte ni autorización, presentando riesgos como el problema del "delegado confundido". Esto es cuando un usuario de bajo privilegio engaña al modelo AI para que solicite una acción delicada al servidor MCP; el servidor ve la petición como legítima desde el modelo confiable y la ejecuta sin verificar si el usuario tiene permisos para esa acción específica. Esto es una escalada de privilegios vía inyección de prompt.

¿Cómo se adopta MCP de forma segura para empresas? El consenso es que el protocolo MCP debe estar envuelto en capas externas centralizadas de gobernanza y seguridad. No se deben exponer los servidores MCP directamente. Se usan API gateways empresariales robustos (como Apogee o similares) delante de ellos, que manejan autenticación sólida, autorizaciones precisas por acción, limitación de tasa, logging y filtrado de entradas potencialmente maliciosas antes de que lleguen al servidor MCP o al LLM. La seguridad no está en MCP, sino alrededor de MCP. Se aprovecha infraestructura de seguridad empresarial madura para proveer un marco seguro necesario.

Resumiendo: Los modelos base son cerebros poderosos, pero necesitan herramientas (a través de estándares como MCP) para actuar en el mundo. Para obtener verdadero valor se requiere definir herramientas con claridad, mantener salidas disciplinadas, señalar errores instructivamente y agregar capas externas de seguridad y gobernanza. Estos principios, desde la nomenclatura hasta la modularidad, son aplicables ampliamente y esenciales para construir sistemas agenticos prácticos y seguros.]

Día # 3: Ingeniería de Contexto, Sesiones y Memoria de trabajo.

Si te interesa construir agentes que recuerdan quién eres, qué te gusta y de qué hablaron la última vez, este documento es el plano a seguir. Nuestra meta aquí es decodificar los elementos clave, la arquitectura necesaria para transformar una simple llamada a un LLM en algo con estado, persistente y, en definitiva, realmente inteligente.

Para lograrlo, hay que entender tres ideas centrales que están ligadas y representan un gran cambio en cómo interactuamos con estos modelos. Primero está la ingeniería de contexto, que es como el controlador maestro. Se trata de gestionar dinámicamente toda la información relevante en la ventana de contexto del LLM cada vez que conversas con él. Segundo, las sesiones. Una sesión es el contenedor para una conversación completa: guarda el historial y la “memoria de trabajo” para esa interacción específica. Y tercero, la memoria, que es el mecanismo de persistencia a largo plazo: permite capturar y consolidar conocimiento a través de muchas sesiones y es la base de la personalización real.

La ingeniería de contexto se considera la base porque ayuda a remediar el talón de Aquiles de los LLM: por defecto son “stateless”. Cada llamada a la API empieza desde cero; el modelo solo es consciente del contexto que le das en ese momento. Construir estado implica armar “paquetes” de información en cada turno. Esa labor de reconstruir el input dinámicamente es mucho más compleja que la ingeniería de prompts tradicional, que suele ser más estática. La ingeniería de contexto implica seleccionar, recortar y adaptar datos, instrucciones, ejemplos y resultados recientes para que cada respuesta esté optimizada al instante.

Uno de los grandes retos es el “context rot”: cuando la ventana de contexto se llena de ruido, la capacidad de razonar del modelo cae. Aquí, la ingeniería de contexto incluye técnicas como resumen, recorte selectivo del historial o mutación dinámica del contexto. Hay que mantener solo lo esencial y comprimir todo lo demás. El ciclo para cada turno significa: buscar recuerdos relevantes, preparar el contexto, invocar al modelo y luego actualizar la memoria de largo plazo, preferentemente de forma asíncrona.

La sesión es el espacio de trabajo para una conversación, específica de un usuario y normalmente tiene dos partes: events (el log cronológico estricto) y state (la memoria de trabajo estructurada, como el carrito de compras en una sesión de ecommerce). Cómo se gestiona esto depende del framework: ADK, por ejemplo, diferencia events y states, pero otros frameworks pueden usar objetos mutables que permiten compresión y resumen del historial en caliente.

Para sistemas multiagente, el white paper distingue entre historia compartida (todo en un solo log, útil para agentes muy coordinados, pero puede ser caótico) y historias separadas para cada agente (más aislado, pero se pierde contexto compartido). De ahí la necesidad de una capa de memoria abstracta que permita compartir conocimientos sintetizados entre agentes aislados.

En producción, la gestión de sesiones tiene implicaciones de privacidad y rendimiento. Es fundamental reducir datos sensibles antes de almacenarlos (por normativas como GDPR) y aplicar políticas de TTL ("time to live"), además de estrategias de compactación para mantener la eficiencia y no saturar la ventana de contexto ni aumentar latencia.

La compactación puede ser tan simple como conservar solo los últimos turnos ("sliding window") o truncar por tokens, o más sofisticados como resumen recursivo usando el propio LLM para comprimir partes viejas del historial. Esto debe hacerse en segundo plano para no afectar la experiencia del usuario.

Sobre memoria, el paper la compara con el asistente personal: maneja datos personalizados (tus preferencias, tus procesos) y se diferencia de la búsqueda RAG, que es como un bibliotecario para hechos generales. La memoria puede incluir datos declarativos (hechos), procedimentales (cómo hacer cosas) y hay que organizarla bien. Arquitectónicamente, suelen usarse vector DB para búsquedas semánticas y grafos de conocimiento para relaciones. La memoria puede ser a nivel de usuario (la más común), de sesión (temporal) o de aplicación (global, pero requiere mucha sanitización para no mezclar usuarios).

La generación de memorias sigue el patrón ETL (extract-transform-load): filtrar, consolidar y luego comparar y fusionar o descartar según relevancia, fuente y confiabilidad, seguido de actualización asíncrona en un store persistente.

La memoria como herramienta es un enfoque útil: el LLM puede decidir cuándo guardar y cuándo buscar algo durante la conversación, logrando autonomía y eficiencia. Para recuperar memorias, lo ideal es combinar criterios de relevancia semántica, recencia e importancia, evitando depender de un solo factor.

Finalmente, la ubicación de memorias en el prompt importa: ponerlas en instrucciones del sistema da mucho peso (bueno para datos estables, pero peligroso si la memoria está equivocada); incluirlas en el historial puede añadir ruido.

Para avalar todo este sistema se requiere testing serio: métricas de generación, recall en retrieval, latencia baja y sobre todo evaluar si realmente mejora la experiencia del

usuario. El marco conceptual es claro: la ingeniería de contexto, sesiones y memoria permiten construir agentes que evolucionan hacia asistentes adaptativos, persistentes y realmente personalizados.]

Día #4 “Calidad de Agentes”:

Aborda la pregunta central: ¿cómo confiar en un agente de IA cuando no puedes predecir exactamente qué hará? Esa es la cuestión de la no determinación. El whitepaper argumenta con fuerza que la calidad no puede ser un añadido tardío, tiene que ser un pilar arquitectónico desde el inicio, no solamente validación final.

Diseñar para la calidad, no solo probarla. Si el problema principal es la impredecibilidad, ¿qué soluciones propone el documento? Tres mensajes clave. El primero es un cambio de mentalidad: la trayectoria es la verdad. Es decir, no solo importa la respuesta final, sino todo el camino que el agente siguió. Incluso si da la respuesta correcta, si llegó por un camino extraño o erróneo, eso sigue siendo un problema de calidad. Fijarse únicamente en el resultado esconde los tropiezos y casi errores.

Por eso, el segundo punto es que necesitas ver la trayectoria. La observabilidad es la base: necesitas herramientas, logs, tracing, métricas para entender el razonamiento del agente. Sin esa ventana, evalúas y depuras a ciegas.

Tercero, la evaluación es un ciclo continuo, no algo que se hace una vez antes de lanzar. Lo llaman el “flywheel” de calidad: todo lo que se aprende cuando el agente falla en el mundo real retroalimenta y mejora el sistema. Es un ciclo sin fin de mejora.

El whitepaper compara la evaluación tradicional (software como camión repartidor, rutas fijas, todo o nada, fácil ver los fallos) contra la evaluación de agentes (como un Fórmula 1: decisiones dinámicas en tiempo real y riesgos más sutiles). Los agentes pueden fallar silenciosamente: la salida parece correcta, pero el contenido es erróneo, sesgado o inútil, erosionando la confianza poco a poco.

Lista modos de fallo específicos de agentes: sesgos algorítmicos (por ejemplo, un agente que discrimina candidatos en selección de CV sin intención), alucinaciones factuales, drift de rendimiento (el mundo cambia y el agente no se adapta, como fraudes nuevos que no detecta) y comportamientos emergentes o no intencionados (agentes que encuentran atajos absurdos para lograr objetivos).

En agentes abiertos al mundo, se añade la complejidad de razonamiento multi-paso, planificación, uso de herramientas, función de memoria y aprendizajes a largo plazo.

Por ello, la meta de la evaluación debe pasar de simple verificación (“¿construimos bien el sistema?”) a validación (“¿construimos el sistema correcto para el usuario?”). Se proponen cuatro pilares de calidad:

- Efectividad: ¿logra el agente el objetivo del usuario, no solo la tarea técnica?
- Eficiencia: ¿lo hace rápido, barato y con un trayecto razonable?
- Robustez: ¿cómo responde ante errores, APIs caídas, instrucciones ambiguas?
- Seguridad y alineación: ética, evitar daños, resistir ataques, cumplir límites de uso y proteger al usuario.

La estrategia de evaluación recomendada es jerárquica de afuera hacia dentro: primero evaluación de caja negra (¿cumple el objetivo global?), y si falla, análisis interno del trayecto y decisiones de razonamiento, uso de herramientas, interpretación de respuestas. En Kaggle ADK, por ejemplo, puedes guardar trayectorias exitosas como “evil cases” para pruebas regresivas automáticas en el futuro.

¿Quién evalúa? Debe ser un sistema híbrido: métricas automáticas (por ejemplo, rouge o bert-score) para velocidad y escala (sirven como alarma temprana en el pipeline de desarrollo), LLM-judge (un modelo avanzado como Gemini compara salidas entre agentes con un criterio claro) para calidad detallada, e incluso agentes evaluadores especializados. Pero la revisión humana sigue siendo esencial para calidad, creatividad y creación del set dorado (golden set) de ejemplo.

Los mejores resultados se logran con buenas UIs de revisión que muestran tanto la conversación como la trayectoria de razonamiento del agente y, en acciones críticas, el humano toma la aprobación final. Esto se conecta con la IA Responsable (RAI): pruebas de “red teaming” para encontrar vulnerabilidades, y sistemas de seguridad como plugins de seguridad (chequeos de inyección de prompt antes y después de cada ejecución).

La base técnica de todo es la observabilidad, con tres pilares: logging (el diario, pasos detallados en formato estructurado, idealmente JSON, con entradas, salidas y razonamiento); tracing (la narrativa, enlazando los eventos para mostrar causas y efectos usando estándares como opentel); y métricas (los KPIs de salud: latencias, errores, costos y calidad, adaptados a roles de operaciones o ciencia de datos).

Grabar todo puede ser costoso; se recomienda muestreo dinámico, registrando todos los errores y solo una fracción de los éxitos, para balancear insight y rendimiento.

El flywheel de calidad de agentes es el gran resumen: define métricas y objetivos, instrumenta la observabilidad, evalúa agentes y trayectorias, y realimenta el aprendizaje de cada interacción (especialmente las fallidas) para mejorar el sistema de manera continua.

Principios clave del whitepaper: la evaluación es pilar arquitectónico, el trayecto es la verdad (no solo el resultado) y el humano sigue siendo el árbitro y el núcleo de la definición de calidad. El futuro será agentico, pero solo con este marco podemos construir agentes realmente fiables y seguros. El llamado es claro: instrumenta y haz evaluables tus agentes desde hoy.